AmritCare

Smart healthcare systems

[2025–2026]

Minor Project Report Submitted to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal towards the partial fulfillment of the degree of

MASTER OF COMPUTER APPLICATION
(COMPUTER TECHNOLOGY AND APPLICATIONS)

Project Guide:

Ms. Shweta Gupta
Asst. Professor
Dept. of CTA

Submitted by:

Pranav Pratap Singh Sisodiya
0801CA251099

SHRI GOVINDRAM SEKSARIA INSTITUTE OF TECHNOLOGY AND SCIENCE, INDORE (M.P.)

A Govt. Aided Autonomous Institute, Affiliated to RGPV, Bhopal

DEPARTMENT OF COMPUTER TECHNOLOGY AND APPLICATION

RECOMMENDATION

We are pleased to recommend that the project work entitled **AmritCare – Smart healthcare systems** submitted by **Pranav Pratap Singh Sisodiya** may be accepted in partial fulfillment of the degree of **Master of Computer Application** of **Shri Govindram Seksaria Institute of Technology and Science, Indore** during the **session 2025–2026.**

Project Guide:

Ms. Shweta Gupta
Asst. Professor
Dept. of CTA

Dr. Sunita Verma
Head of Department
Dept. of CTA

SHRI GOVINDRAM SEKSARIA INSTITUTE OF TECHNOLOGY AND SCIENCE, INDORE (M.P.)

A Govt. Aided Autonomous Institute, Affiliated to RGPV, Bhopal

DEPARTMENT OF COMPUTER TECHNOLOGY AND APPLICATION

CERTIFICATE

This is to certify that the project report entitled **AmritCare** submitted by **Pranav Pratap Singh Sisodiya** student of I year **MCA** in the year **2025-26** of the Institute is a satisfactory account of their MINOR Project work based on the syllabus.

Internal Examiner **External Examiner**

Date:- _____ Date:- _____

Declaration

We **0801CA251091 Nitish Kumar, 0801CA251099 Pranav Pratap Singh Sisodiya, 0801CA251113 Ramdev Thapak and 0801CA251143 Uttam Carpanter**, student of MCA I year in the session 2025-26, hereby declare that the work submitted **AmritCare** is our own work conducted under the supervision of **Ms. Shweta Gupta, MCA.**

We further declare that to the best of our knowledge, this work does not contain any part of any work which has been submitted for the award of any degree or any other work either in this University or in any other University without proper citation.

Nitish Kumar	0801CA251091	Signature _____
Pranav Pratap Singh Sisodiya	0801CA251099	Signature _____
Ramdev Thapak	0801CA251113	Signature _____
Uttam Carpenter	0801CA251143	Signature _____

Acknowledgement

We take this opportunity to express our sincere gratitude to all those who guided and supported us throughout the development of this project.

We are deeply thankful to our Project Guide, **Ms Shweta Gupta**, Department of Computer Technology and Application, SGSITS, Indore, for their invaluable guidance, continuous encouragement, and constructive suggestions at every stage of this work.

We extend our heartfelt thanks to the Head of Department, Department of Computer Technology and Application, SGSITS, Indore, for providing the necessary facilities and an encouraging academic environment.

We are thankful to all the faculty members of the Department for their constant support and motivation.

Finally, we would like to thank our family and friends for their patience and moral support throughout the project.

0801CA251091	Nitish Kumar
0801CA251099	Pranav Pratap Singh Sisodiya
0801CA251113	Ramdev Thapak
0801CA251143	Uttam Carpenter

Abstract

Smart healthcare systems are essential infrastructure for managing medical resources efficiently in rapidly growing metropolitan cities. Handling hospital availability, patient appointments, ambulance coordination, and administrative oversight manually leads to delays, resource mismanagement, and reduced quality of care.

AmritCare is a full-stack web application designed to digitize and streamline city-wide healthcare management. It provides a centralized platform connecting patients, hospitals, and emergency services in real time. The system is built around three primary user roles, each with dedicated functionalities:

1. **Patient Module** – General users can search hospitals based on location and specialization, view real-time availability of beds, ICU units, and doctors, and book appointments instantly. The system ensures a smooth and user-friendly experience, including a voice assistant powered by the Web Speech API for hands-free navigation.
2. **Hospital Administration** – Hospital administrators manage their institution's resources through a dedicated dashboard. They can update availability of beds, ICU units, and doctors dynamically, as well as view and process patient bookings. Server-side validation prevents overbooking and ensures data consistency.
3. **City Administration** – The city administrator has a centralized control panel providing a comprehensive overview of all hospitals and healthcare resources. This includes system-wide monitoring, resource analytics, and decision-support insights through interactive visual dashboards.

Key features of the system include real-time resource tracking using live database queries, role-based authentication with session management, asynchronous updates using AJAX for seamless user interaction, an ambulance tracking interface with Canvas-based simulation for visualizing movement across the city, and an analytics dashboard powered by Chart.js.

The application follows the MVC (Model-View-Controller) architecture and uses the DAO (Data Access Object) design pattern for structured database interaction. It is developed using Java Servlets, JSP, MySQL, and Apache Tomcat. The database consists of multiple interrelated tables managing users, hospitals, doctors, beds, ICU units, bookings, and ambulance data.

AmritCare aims to provide a scalable, efficient, and beginner-friendly solution for smart city healthcare management, demonstrating practical full-stack Java development concepts applicable to real-world urban healthcare systems.

Contents

Chapter 1 INTRODUCTION

1.1 Preamble

The healthcare system is one of the most critical components of any city's infrastructure. Rapid urbanization, population growth, and increasing demand for quality medical services have exposed serious gaps in traditional healthcare management. Patients often visit hospitals without knowing whether beds, ICU units, or specialist doctors are available, leading to wasted time, panic, and sometimes life-threatening delays.

AmritCare — an Intelligent City-Wide Smart Healthcare Management System — is a centralized, web-based platform built to address these challenges. It acts as a digital bridge between hospitals, patients, and ambulance services across an entire city, providing real-time resource visibility and streamlined booking.

1.2 Need of the Project

The healthcare ecosystem in a city like Bhopal, Madhya Pradesh, faces significant coordination problems:

- Patients travel to hospitals without knowing real-time bed or doctor availability.
- Hospital resources (beds, ICUs) are managed manually, leading to errors and miscommunication.
- Ambulance dispatch is uncoordinated, with no live tracking or nearest-hospital routing.
- City health administrators lack a consolidated view of system-wide resource utilization.
- Emergency situations worsen due to lack of real-time information and digital coordination.

AmritCare directly addresses all of these pain points by providing a centralized digital platform accessible to all stakeholders.

1.3 Problem Statement

Cold storage facility administrators lack a centralized digital platform to manage chamber inventory, customer records, and bookings. Existing manual processes cause operational delays, billing inaccuracies, and loss of historical data. There is a need for a system that can handle the complete lifecycle of a cold storage booking — from slot reservation to checkout — with accurate, automated billing and record keeping.

1.4 Objectives

The objectives of ColdVault are as follows:

- Develop a real-time hospital resource tracking system covering beds, ICUs, and doctors.
- Implement a multi-role authentication and authorization system (Patient, Hospital Admin, City Admin).
- Build an instant booking system with overbooking prevention logic.
- Create an ambulance tracking interface with Canvas-based live simulation.
- Integrate a Web Speech API–based voice assistant for hands-free navigation.
- Provide analytical dashboards with Chart.js visualizations for city administrators.
- Follow MVC and DAO design patterns for clean, maintainable code.

1.5 Proposed Approach

AmritCare is developed as a Java-based web application using the following approach:

- **Frontend:** HTML5, CSS3, JavaScript with responsive design principles and AJAX for dynamic updates.
- **Backend:** Java Servlets acting as controllers, JSP pages as views.
- **Database:** MySQL with a well-normalized schema of seven related tables.
- **Connectivity:** JDBC with a Singleton connection class (DBConnection).
- **Design Pattern:** MVC architecture with DAO layer for database abstraction.
- **Deployment:** Apache Tomcat 10.1 servlet container.

1.6 Organization of Report

The report is organized into the following chapters:

- Chapter 1 — Introduction: Background, need, objectives, and proposed approach.
- Chapter 2 — Literature Review: Survey of existing healthcare management systems.
- Chapter 3 — Requirements Analysis: Functional and non-functional requirements.
- Chapter 4 — Analysis and Design: Architecture, database design, and system diagrams.
- Chapter 5 — Implementation: Key code components and screenshot walkthroughs.

- Chapter 6 — Result and Discussion: Testing, output screenshots, and performance.
- Chapter 7 — Conclusion and Future Scope: Summary and planned enhancements.

Chapter 2 LITERATURE REVIEW

2.1 Overview of Healthcare Information Systems

Healthcare Information Systems (HIS) have been studied and deployed globally since the 1980s. Early systems were standalone hospital management tools managing patient registration and billing. With the advent of web technologies in the 2000s, these evolved into networked systems capable of serving multiple departments within a hospital. The emergence of smart city initiatives has driven a new wave of city-wide, interoperable health platforms.

2.2 Survey of Existing Systems

Several healthcare platforms have been developed and studied in the literature:

- **Hospital Management Systems (HMS)** such as Bahmni, OpenMRS, and eSanjeevani focus on electronic medical records and patient registration but lack real-time resource tracking across multiple hospitals.
- **National Health Stack (NHS Digital India):** Provides a framework for digital health records but is not designed for real-time bed/ICU booking by patients.
- **COVID-19 Bed Availability Portals** (e.g., MP's corona.mp.gov.in): Demonstrated the value of live bed tracking but were emergency-only solutions with limited features.
- **Google Health / Practo:** Provide doctor appointment booking but do not address hospital resource management, ICU tracking, or ambulance coordination.

2.3 Comparative Analysis

Table 2.1: Comparative Analysis of Existing Healthcare Systems

Feature	**AmritCare**	**Practo**	**eSanjeevani**	**HMS (Generic)**
Real-time Bed Tracking	Yes	No	No	No
ICU Availability	Yes	No	No	Partial
Doctor Appointment	Yes	Yes	Yes	Yes
Ambulance Tracking	Yes (Sim)	No	No	No
Voice Assistant	Yes	No	No	No
City Admin Dashboard	Yes	No	Partial	No
Open Source / Free	Yes	No	Yes	Paid

The comparative analysis clearly shows that AmritCare offers a unique combination of features not available in any single existing solution.

2.4 Research Gap

Existing systems are either too specialized (single hospital only) or too generic (no real-time resource tracking). None offer a complete, open, city-wide platform with voice navigation, ambulance tracking, and multi-role access in a single cohesive system — which is precisely the gap AmritCare fills.

Chapter 3 ABOUT SYSTEM REQUIREMENTS

3.1 Functional Requirements

3.1.1 Patient Module

- FR-P1: Register a new patient account with name, email, phone, and password.
- FR-P2: Log in and log out with session management.
- FR-P3: Search hospitals by name, location, or specialization using AJAX.
- FR-P4: View real-time availability of beds, ICU units, and doctors per hospital.
- FR-P5: Book a bed, ICU slot, or doctor appointment with form validation.
- FR-P6: View personal booking history with status (Pending/Confirmed/Cancelled/Completed).
- FR-P7: Use voice commands to navigate the portal hands-free.

3.1.2 Hospital Admin Module

- FR-H1: Login with hospital-admin credentials.
- FR-H2: View all patient bookings for the respective hospital.
- FR-H3: Confirm, cancel, or mark bookings as completed.
- FR-H4: Add, remove, and toggle availability of doctors.
- FR-H5: Update per-unit status of individual beds and ICU units (Available/Occupied/Maintenance).

3.1.3 City Admin Module

- FR-A1: View city-wide statistics: total hospitals, patients, bookings, and ambulances.
- FR-A2: Add new hospital records with location, GPS coordinates, and capacity.
- FR-A3: Deactivate or reactivate hospitals.
- FR-A4: View all bookings across all hospitals.
- FR-A5: Access analytics dashboard with Chart.js graphs.

3.1.4 Ambulance Module

- FR-AM1: Display live ambulance fleet on a Canvas-based simulated city map.

- FR-AM2: Show ambulance status: Available, On Call, Returning.
- FR-AM3: Display nearest hospitals with available beds and ICU counts.
- FR-AM4: Provide a prominent emergency call (108) button.

3.2 Non-Functional Requirements

Table 3.1: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Page load time $<$ 3 seconds on standard broadband.
NFR-02	Security	All pages behind authentication; no direct URL access without login.
NFR-03	Usability	Responsive design; usable on desktop, tablet, and mobile browsers.
NFR-04	Reliability	No overbooking; DB constraints prevent concurrent duplicate bookings.
NFR-05	Maintainability	MVC + DAO patterns ensure separation of concerns and easy extension.
NFR-06	Scalability	System can support additional hospitals and cities with minimal changes.

3.3 Hardware and Software Requirements

3.3.1 Development Environment

- Processor: Intel Core i5 or equivalent (2 GHz+)
- RAM: 8 GB minimum
- Storage: 10 GB free disk space
- OS: Windows 10/11

3.3.2 Software Requirements

- JDK: Java Development Kit 8 or above
- Server: Apache Tomcat 10.1
- Database: MySQL Server 8.0+
- IDE: Eclipse IDE for Enterprise Java / IntelliJ IDEA
- JDBC Driver: mysql-connector-j-8.x.x.jar

- Browser: Google Chrome (for Voice API support)

Chapter 4 ANALYSIS AND DESIGN / METHODOLOGY

4.1 System Architecture

AmritCare follows a classic three-tier web architecture:

- **Presentation Tier (Client):** HTML5, CSS3, JavaScript running in the browser. JSP pages render dynamic HTML. Voice Assistant and Canvas ambulance simulation run client-side.
- **Application Tier (Server):** Apache Tomcat hosts Java Servlets (controllers). Servlets process HTTP requests, interact with DAO layer, and forward responses to JSP views.
- **Data Tier (Database):** MySQL 8.0 stores all persistent data across seven normalized tables. JDBC provides the communication bridge between Java and MySQL.

The overall data flow:

Browser → HTTP Request → Servlet → DAO → MySQL → ResultSet → Model → JSP → HTTP Respons

4.2 Technology Stack

Table 4.1: Technology Stack

Layer	**Technology**	**Purpose**
Frontend	HTML5, CSS3, JS (ES6)	UI rendering and interactivity
Voice	Web Speech API	Browser-native speech recognition
Animation	Canvas API / CSS	Ambulance simulation, page animations
Charts	Chart.js 4 (CDN)	Admin analytics visualizations
AJAX	Fetch API	Dynamic hospital search without reload
Backend	Java Servlet 4.0, JSP	Controller and view layer
Database	MySQL 8.0	Persistent storage
Connectivity	JDBC (mysql-connector-j)	Java-to-MySQL communication
Server	Apache Tomcat 9	Servlet container / web server

4.3 UML Diagrams

4.3.1 Use Case Diagram

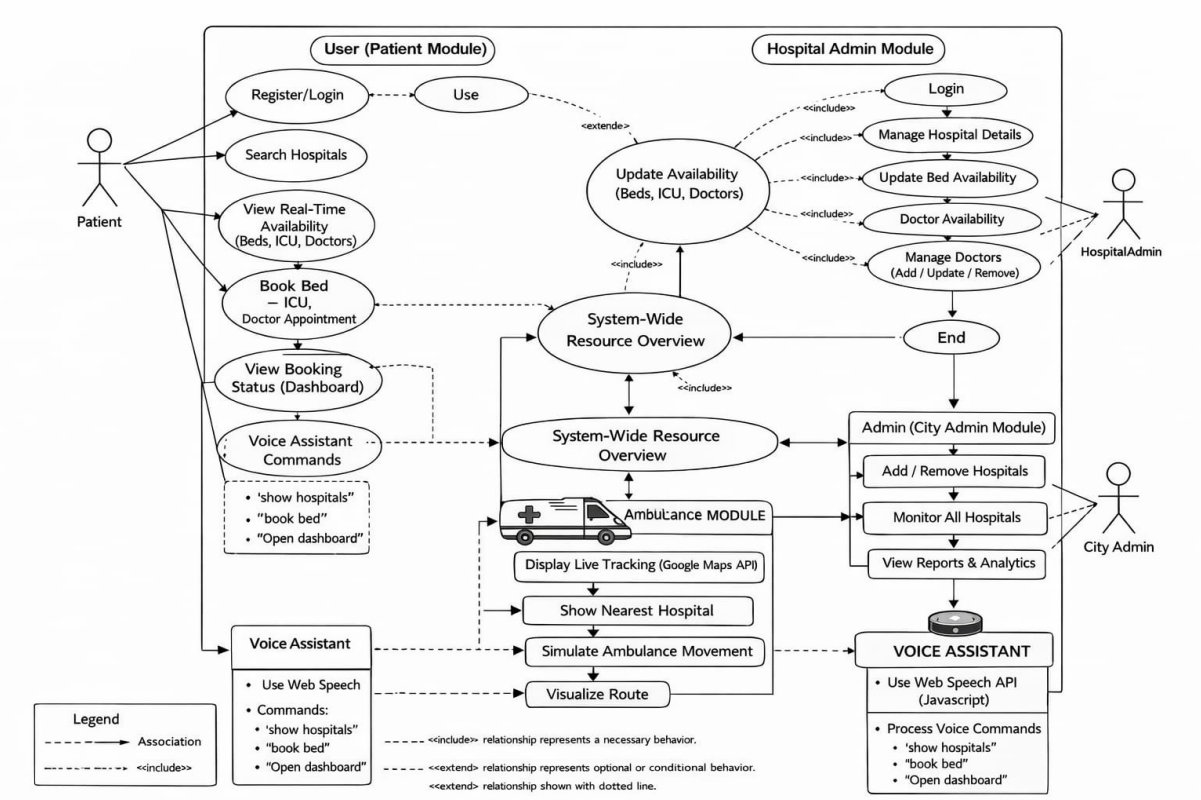

Figure 4.1: Use Case Diagram – AmritCare

4.3.2 E – R Diagram

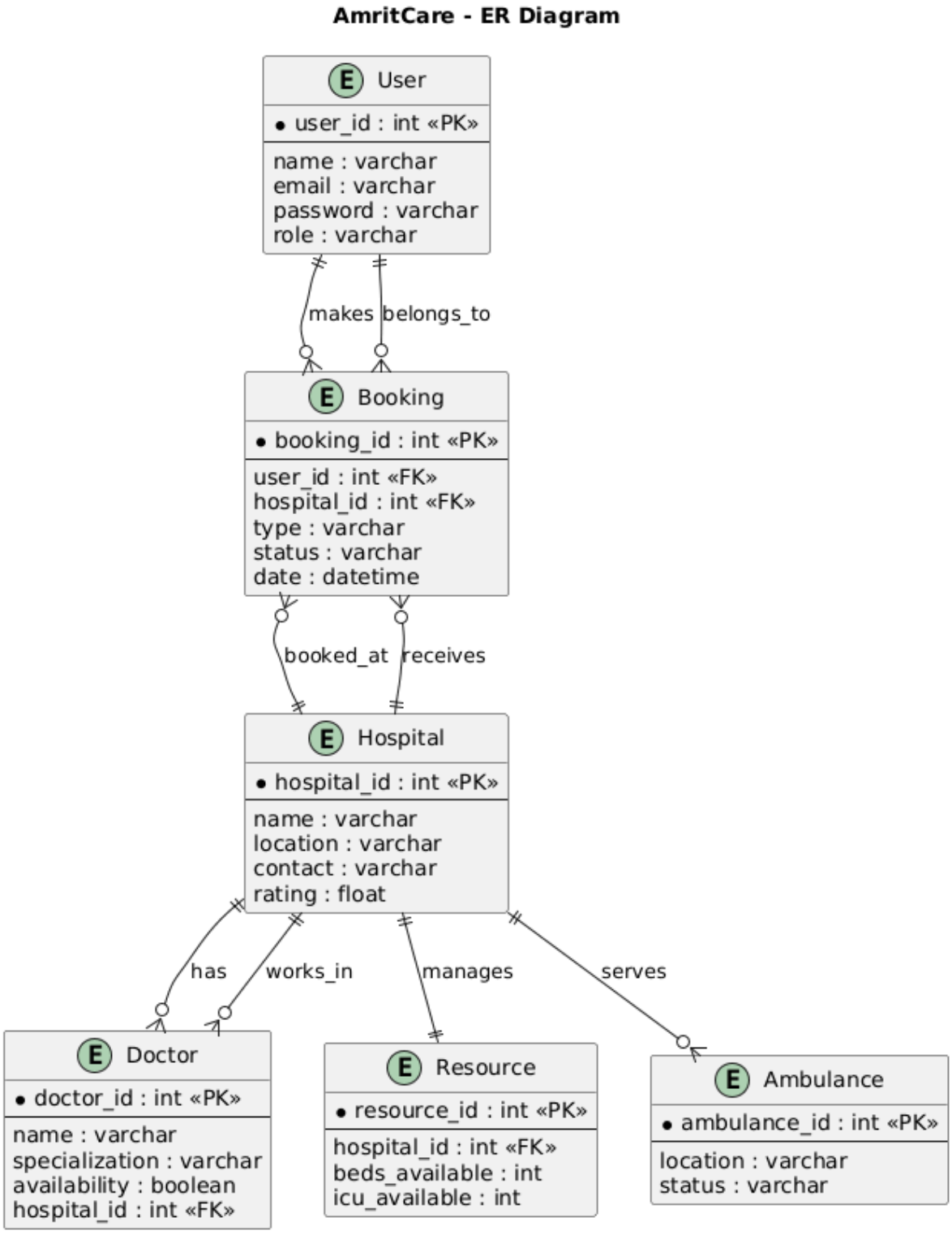

Figure 4.2: Entity–Relationship Diagram

4.3.3 Data Flow Diagram

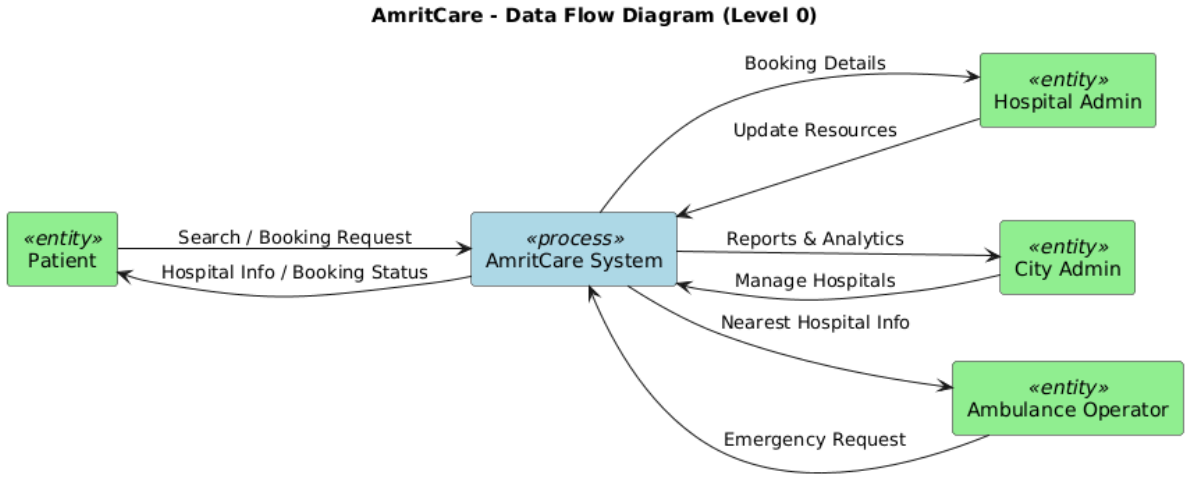

Figure 4.3: Data Flow Diagram

4.3.4 Component Diagram

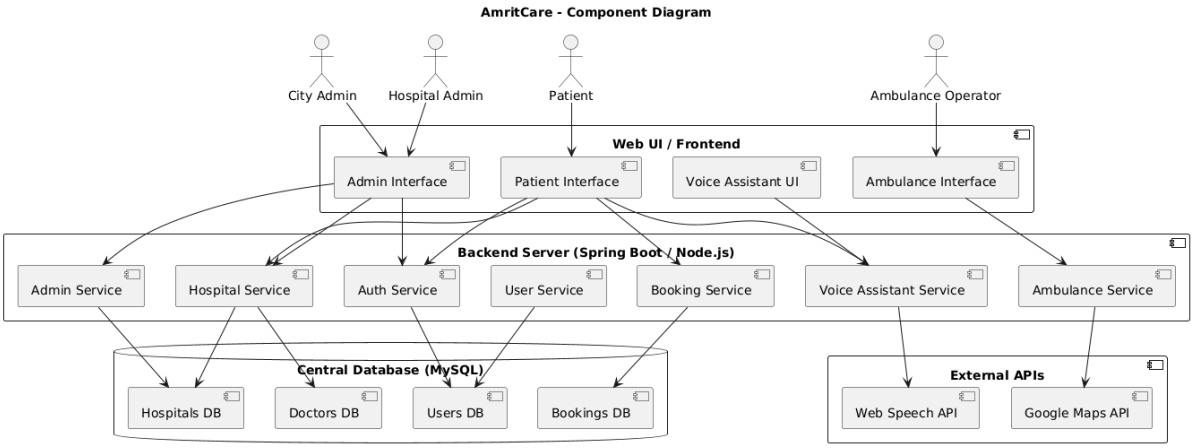

Figure 4.4: Component Diagram

4.3.5 CRC Diagram

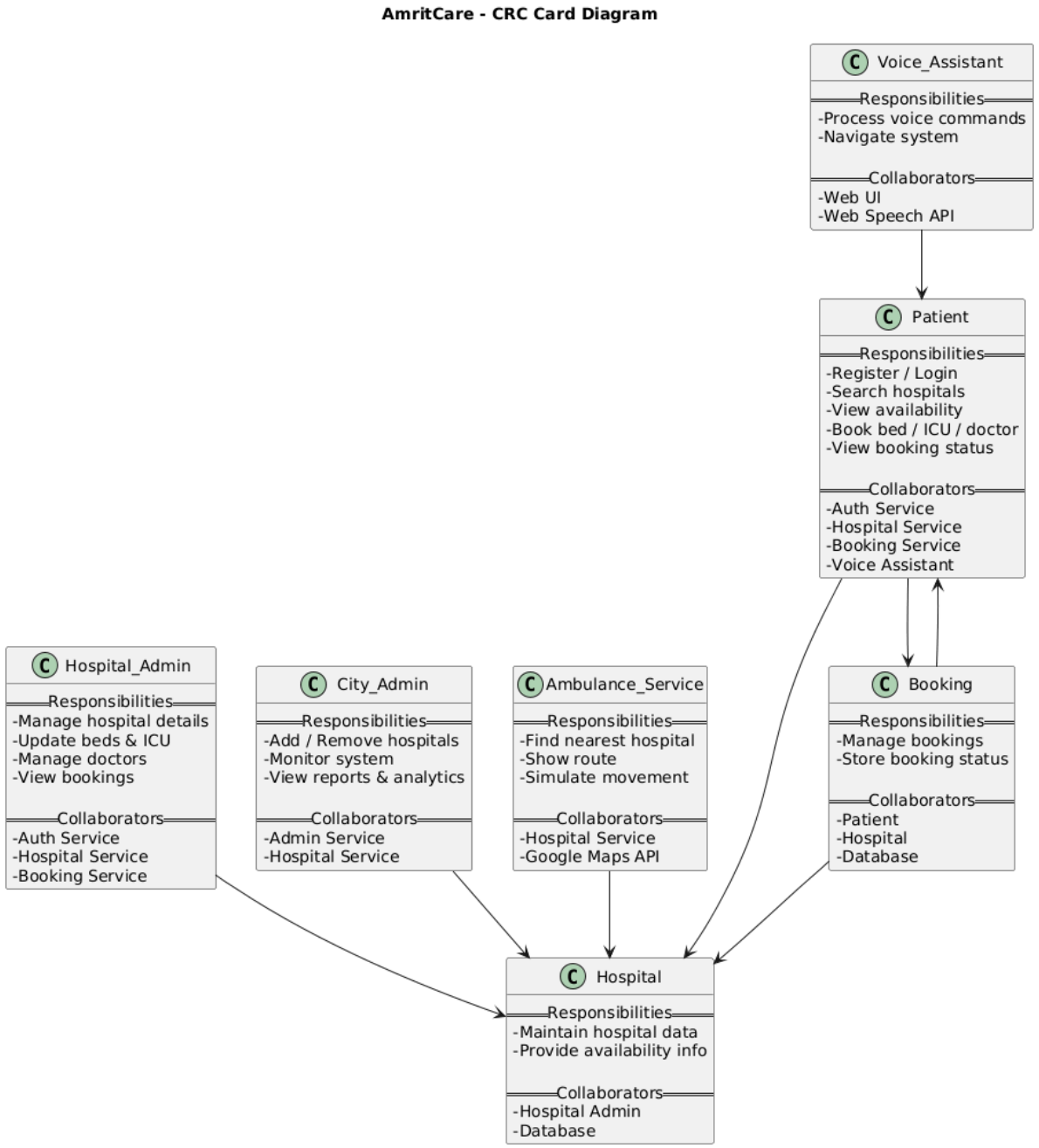

Figure 4.5: CRC Diagram

4.3.6 Collaboration Diagram

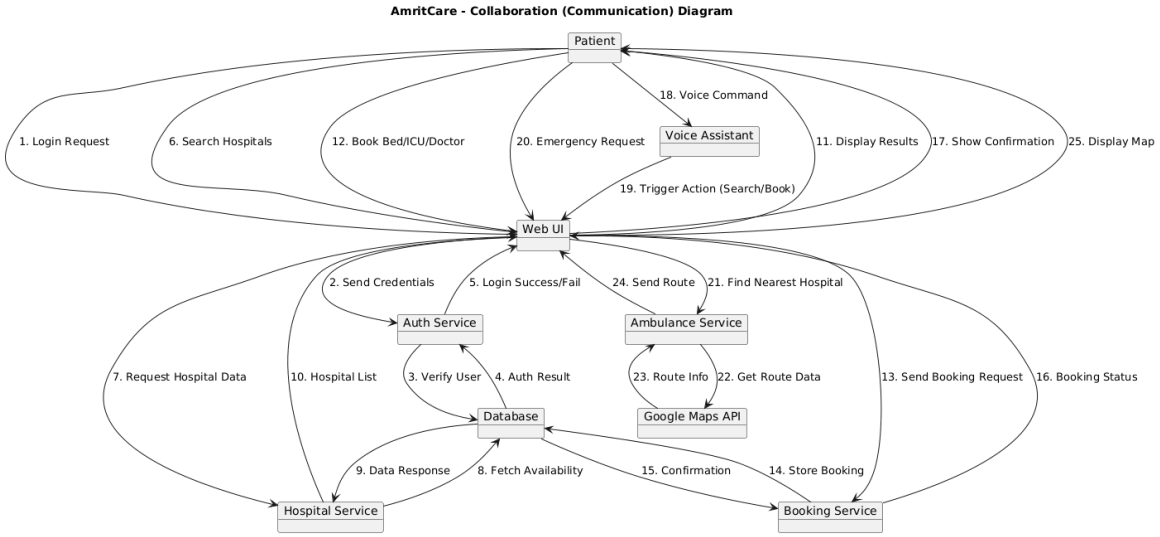

Figure 4.6: Collaboration Diagram

4.3.7 Deployment Diagram

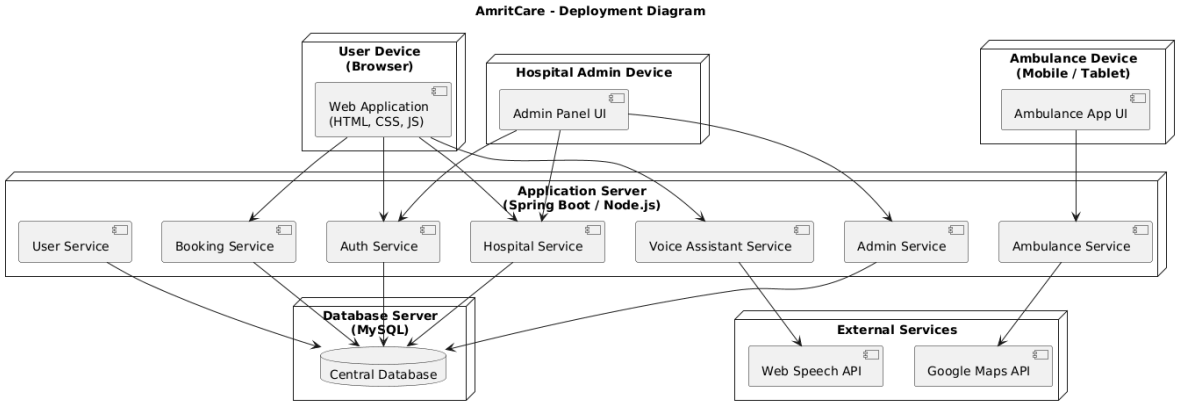

Figure 4.7: Deployment Diagram

4.3.8 State Chart Diagram

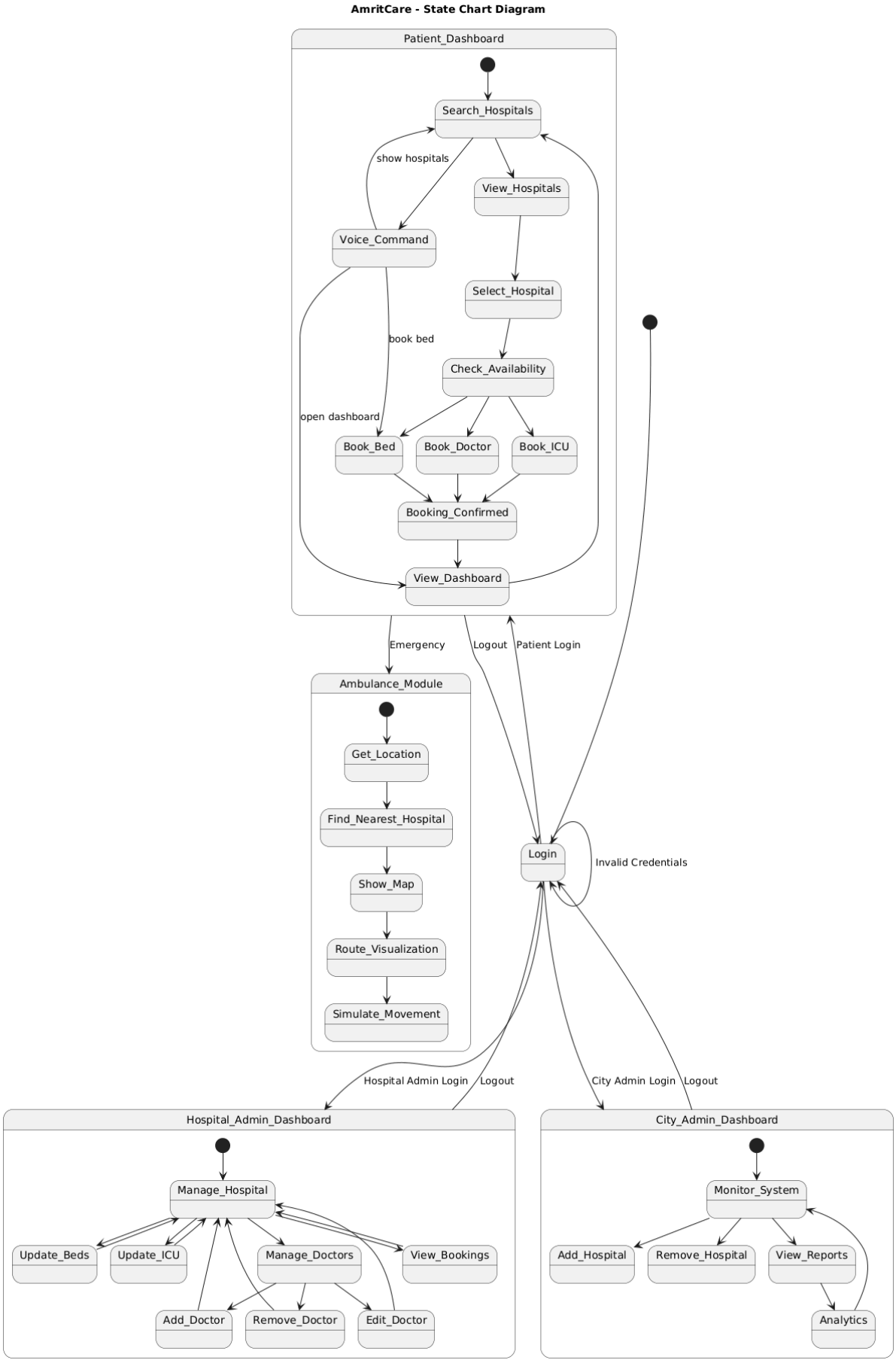

Figure 4.8: State Chart Diagram

4.3.9 Object Diagram – 1

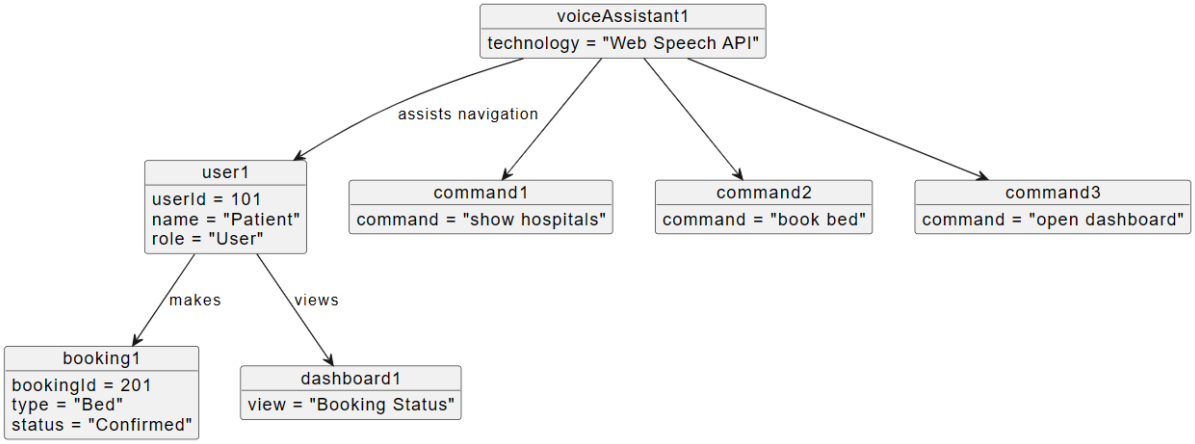

Figure 4.9: Object Diagram 1

4.3.10 Object Diagram – 2

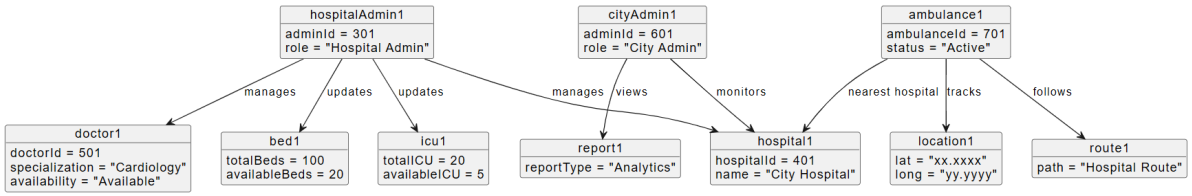

Figure 4.10: Object Diagram 2

4.3.11 Sequence Diagram

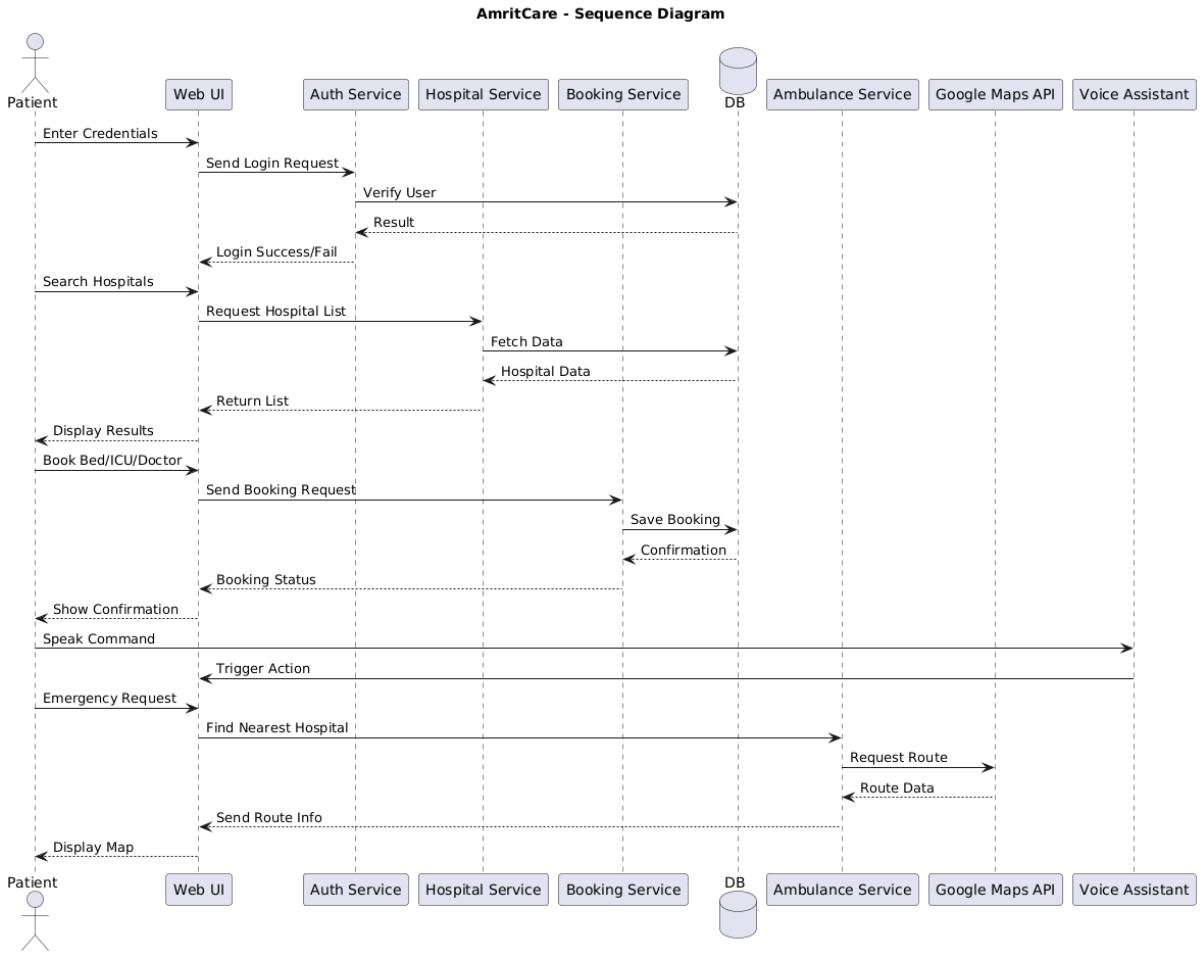

Figure 4.11: Sequence Diagram

4.3.12 Activity Diagram

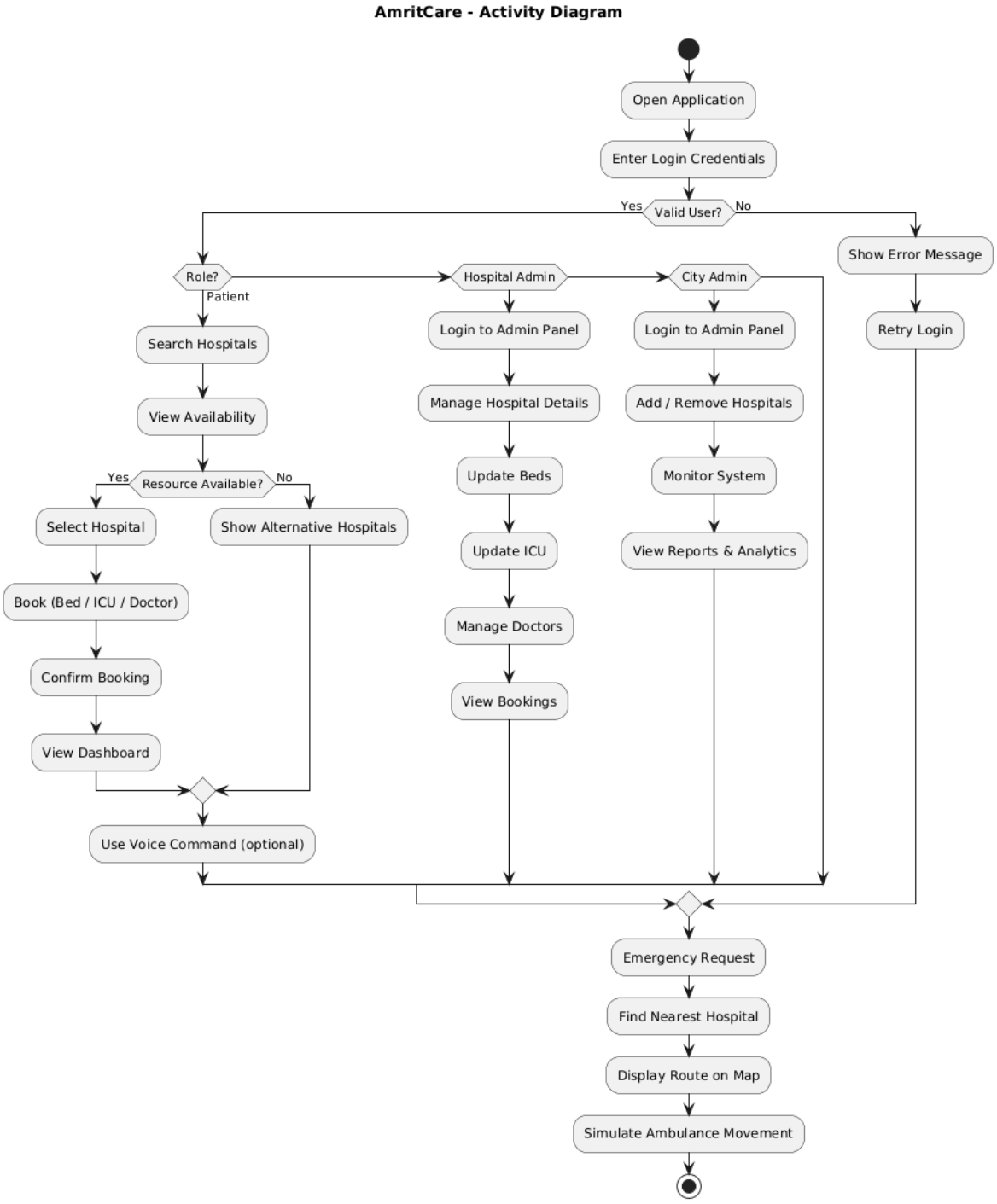

Figure 4.12: Activity Diagram

4.3.13 Class Diagram

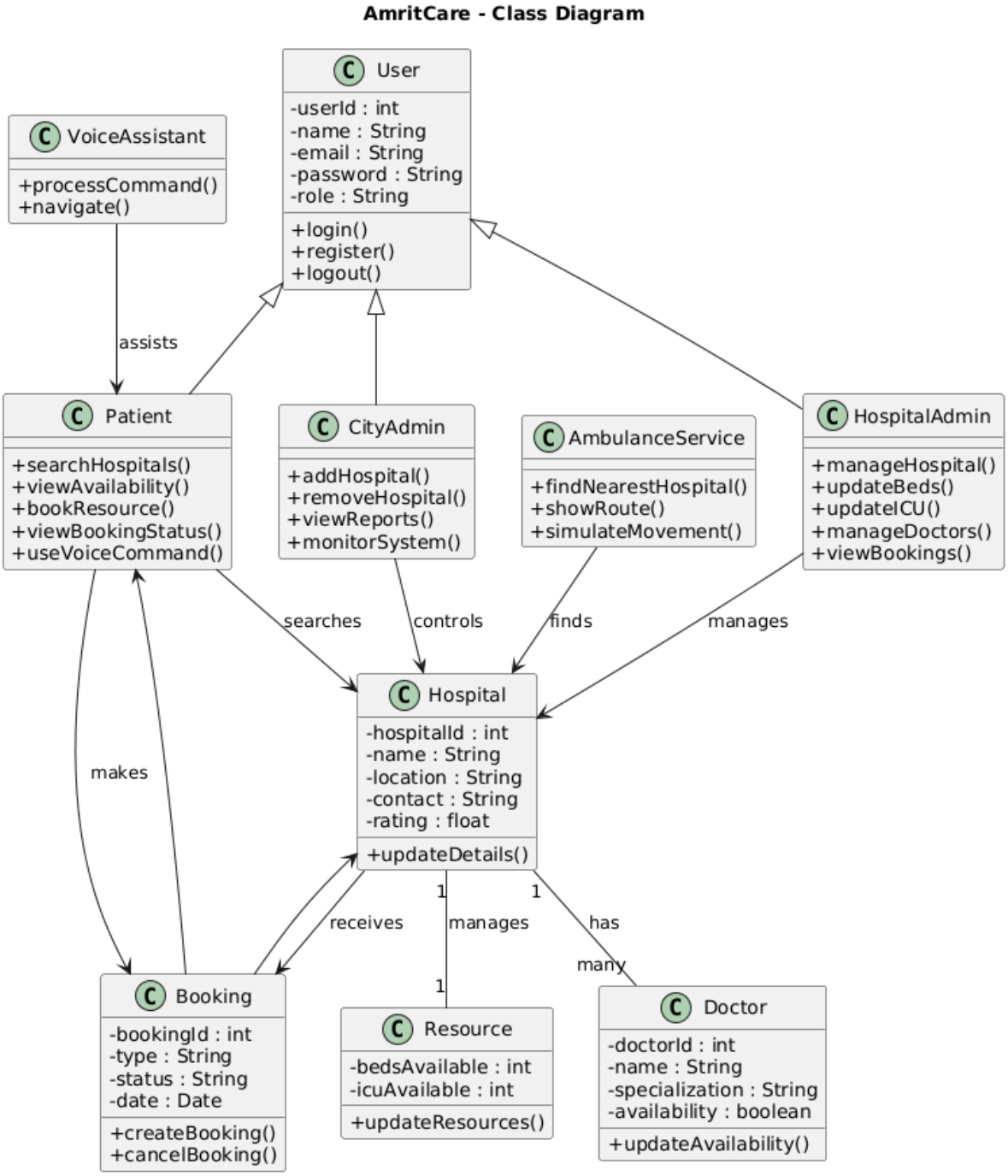

Figure 4.13: Class Diagram

Chapter 5 IMPLEMENTATION / EXPERIMENTATION

5.1 Frontend Implementation

The frontend of AmritCare is designed using JSP, HTML5, CSS3, and JavaScript to provide an interactive and responsive user interface. JSP is used as the view layer in the MVC architecture, dynamically rendering data received from the backend.

The UI is styled using a centralized stylesheet (**style.css**) containing 650+ lines, ensuring consistent design, responsive layouts, and modern components such as cards, dashboards, grids, and navigation panels.

JavaScript (**main.js**) handles all client-side interactivity, including:

- AJAX-based data fetching using the Fetch API for real-time updates (hospital search, doctor lists, resource status)
- DOM manipulation for dynamically rendering hospital cards and booking data
- Form validation for user inputs before submission
- UI animations and transitions for better user experience

The system also integrates advanced browser features:

- **Voice Assistant:** Implemented using Web Speech API for voice-based navigation and commands
- **Chart Visualizations:** Implemented using Chart.js for displaying analytics such as booking status, hospital resources, and usage trends
- **Canvas-Based Simulation:** HTML5 Canvas is used to render ambulance tracking with animated movement

The frontend ensures seamless interaction without full page reloads by combining JSP rendering with AJAX-based partial updates.

5.2 Backend Implementation

The backend of AmritCare is developed using Java Servlets, following the MVC (Model-View-Controller) architecture to separate concerns and improve maintainability.

- **Controller Layer (Servlets):** Servlets handle HTTP requests, process input data, and

control application flow. Examples include LoginServlet, HospitalServlet, and BookingServlet.

- **Model Layer (POJOs):** Java classes represent entities such as User, Hospital, Booking, Doctor, Bed, ICU, and Ambulance. These classes encapsulate data and business logic.
- **DAO Layer (Data Access Object):** DAO classes manage all database operations using JDBC. PreparedStatements are used for secure query execution and to prevent SQL injection.
- **Database Connectivity:** A Singleton-based DBConnection utility ensures efficient reuse of database connections using MySQL.
- **Session Management:** HttpSession is used to maintain user login state and enforce role-based access control across different modules.
- **Validation & Business Logic:** Server-side validation ensures correct input handling, prevents overbooking, and maintains data integrity using constraints and checks.

5.3 Additional Implementation Features

5.3.1 Security Implementation

The system includes basic security mechanisms such as:

- Session-based authentication and authorization
- Input validation on both client and server side
- Use of PreparedStatements to prevent SQL injection
- Restricted access to protected JSP pages via session checks

5.3.2 Database Design and Integration

The MySQL database is designed with multiple interrelated tables: Users, Hospitals, Doctors, Beds, ICU Units, Bookings, and Ambulances.

Relationships are enforced using primary and foreign keys, ensuring referential integrity. The schema supports efficient querying for real-time data display and analytics.

5.3.3 Real-Time Data Handling

The system uses AJAX and periodic updates to simulate real-time behavior:

- Live hospital resource availability
- Dynamic doctor loading based on hospital selection

- Continuous ambulance movement updates
- Instant UI refresh without page reload

5.3.4 Performance Optimization

To ensure smooth performance:

- Reusable database connections via Singleton pattern
- Efficient SQL queries with indexing (where applicable)
- Minimal page reloads using AJAX
- Lightweight frontend with optimized DOM updates

5.3.5 User Experience Enhancements

The system focuses on usability and accessibility:

- Clean dashboard layouts for each user role
- Color-coded status indicators (e.g., booking status)
- Voice-enabled navigation for accessibility
- Interactive charts and visual feedback
- Responsive design for different screen sizes

Chapter 6 RESULT AND DISCUSSION

6.1 System Testing

6.1.1 Unit Testing

Each DAO method was tested individually against the MySQL test database. PreparedStatements were verified to prevent SQL injection. Edge cases tested: empty search keyword, booking when no beds available, duplicate email registration.

6.1.2 Integration Testing

The end-to-end flow was tested: User registers → logs in → searches hospitals → books doctor → hospital admin confirms → city admin views in reports. All session attributes, request forwarding, and database state changes were verified.

6.1.3 Test Cases

Table 6.1: Test Cases Summary — All 9 test cases passed successfully

TC	Test Case	Input	Expected Output	Status
TC01	Valid Patient Login	patient@amritcare.in / patient123	Redirects to dashboard.jsp	PASS
TC02	Invalid Login	wrong@email.com / wrongpw	Error message on login.jsp	PASS
TC03	Register Duplicate Email	Existing email address	Error: Email already registered	PASS
TC04	Book Doctor Appointment	Valid hospital, doctor, date	Booking created, shows in dashboard	PASS
TC05	Hospital Admin Confirm Booking	Click Confirm on pending booking	Status changes to Confirmed	PASS
TC06	Voice Command: 'show hospitals'	Speak into microphone	Navigates to hospitals.jsp	PASS
TC07	Bed Status Update	Change dropdown to Occupied	DB updated, page refreshed	PASS
TC08	Ambulance Animation	Open ambulance.jsp	Dots move on canvas every 2s	PASS
TC9	Unauthorized Access	Access admin_dashboard.jsp without login	Redirected to login.jsp	PASS

6.2 Performance Observations

Table 6.2: System Performance Results — All metrics are within acceptable thresholds

Metric	Observed	Target
Home page load time	$\sim$1.2 seconds	$<$ 3 seconds
Hospital search response	$\sim$350ms	$<$ 1 second
Voice command recognition	$\sim$1–2 seconds	$<$ 3 seconds
Booking form submission	$\sim$400ms	$<$ 2 seconds
Admin reports page with charts	$\sim$1.8 seconds	$<$ 4 seconds

6.3 Discussion

The AmritCare system successfully demonstrated all core objectives. The MVC architecture proved effective in keeping the codebase clean and extensible. The DAO pattern allowed database operations to be tested independently. The AJAX-based search delivered a smooth user experience without page reloads. The Web Speech API voice assistant worked reliably in Chrome and Edge browsers.

The ambulance Canvas simulation, while not connected to real GPS hardware, demonstrates the technical feasibility of the tracking module and can be connected to a real-time GPS API with minimal changes. The Chart.js analytics provided meaningful visual insights for city administrators.

Chapter 7 CONCLUSION AND SCOPE OF FUTURE WORK

7.1 Conclusion

AmritCare — Intelligent City-Wide Smart Healthcare Management System has been successfully designed, developed, and tested as a complete full-stack Java web application. The project demonstrates the power of modern web technologies and software engineering patterns in solving a real-world, high-impact problem.

The system achieves all stated objectives: real-time hospital resource tracking, role-based multi-user access, instant booking with validation, voice-navigated interface, ambulance tracking simulation, and city-wide analytics. It serves as a proof-of-concept for a smart city healthcare platform that could be deployed by municipal health authorities.

The project provided hands-on experience with Java Servlet, JSP, JDBC, MySQL, MVC architecture, DAO design pattern, AJAX, Canvas API, Chart.js, and the Web Speech API — covering virtually the entire spectrum of a full-stack Java web development curriculum.

7.2 Scope of Future Work

The following enhancements are planned for future iterations of AmritCare:

- **Password Security:** Implement BCrypt hashing for stored passwords instead of plain text.
- **SMS/Email Notifications:** Send booking confirmation alerts via Twilio (SMS) or JavaMail (email).
- **Mobile App:** Develop a companion Android app using Android Studio / React Native.
- **Payment Gateway:** Integrate Razorpay or PayU for online consultation fee collection.
- **Electronic Medical Records (EMR):** Add patient health history, prescriptions, and lab report storage.
- **AI-Based Doctor Recommendation:** Use ML models to suggest doctors based on patient symptoms.
- **Multi-City Support:** Extend the system to support multiple cities with configurable admin hierarchies.

- **REST API Layer:** Convert servlets to a RESTful API (JAX-RS / Spring Boot) for better scalability and mobile support.
- **Real-Time Updates:** Integrate WebSockets for live push notifications on resource availability changes.

Chapter A Appendix A: Screenshots

This section includes key screenshots of the AmritCare application illustrating major functionalities and user interfaces.

A.1 Login Page Interface

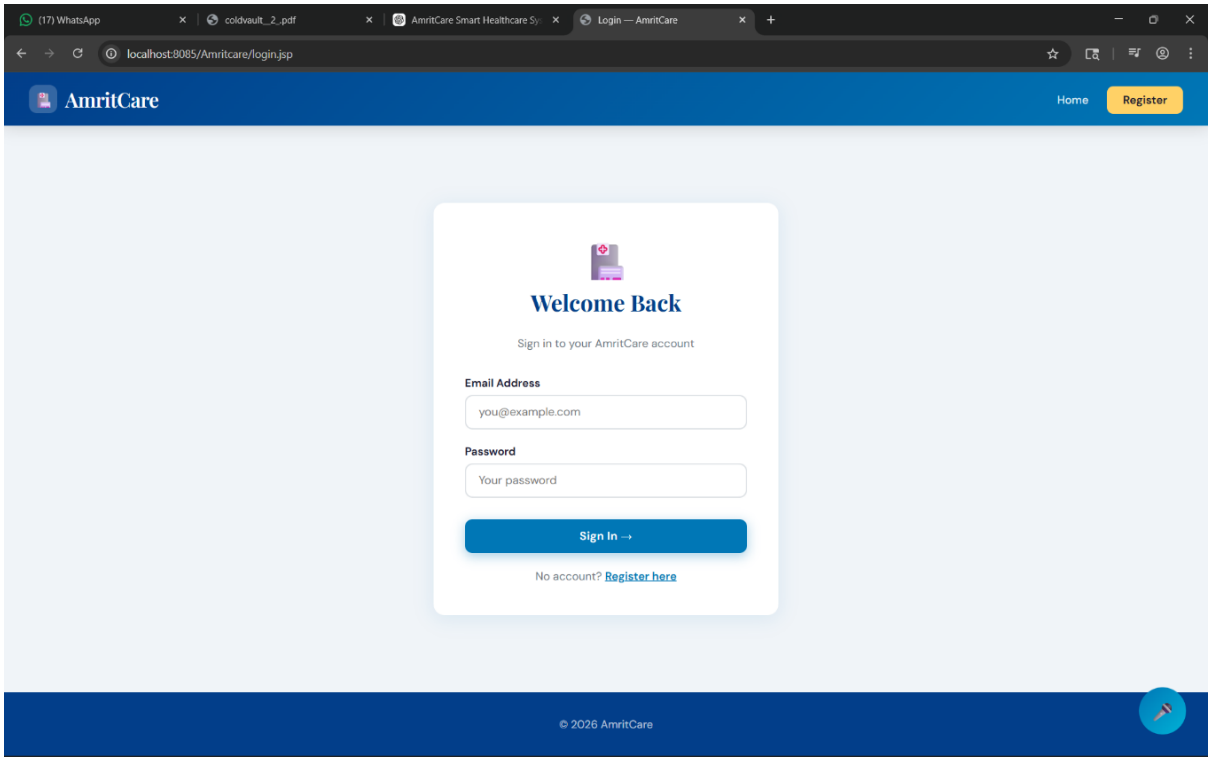

Figure A.1: Login Page Interface of AmritCare

A.2 HomePage

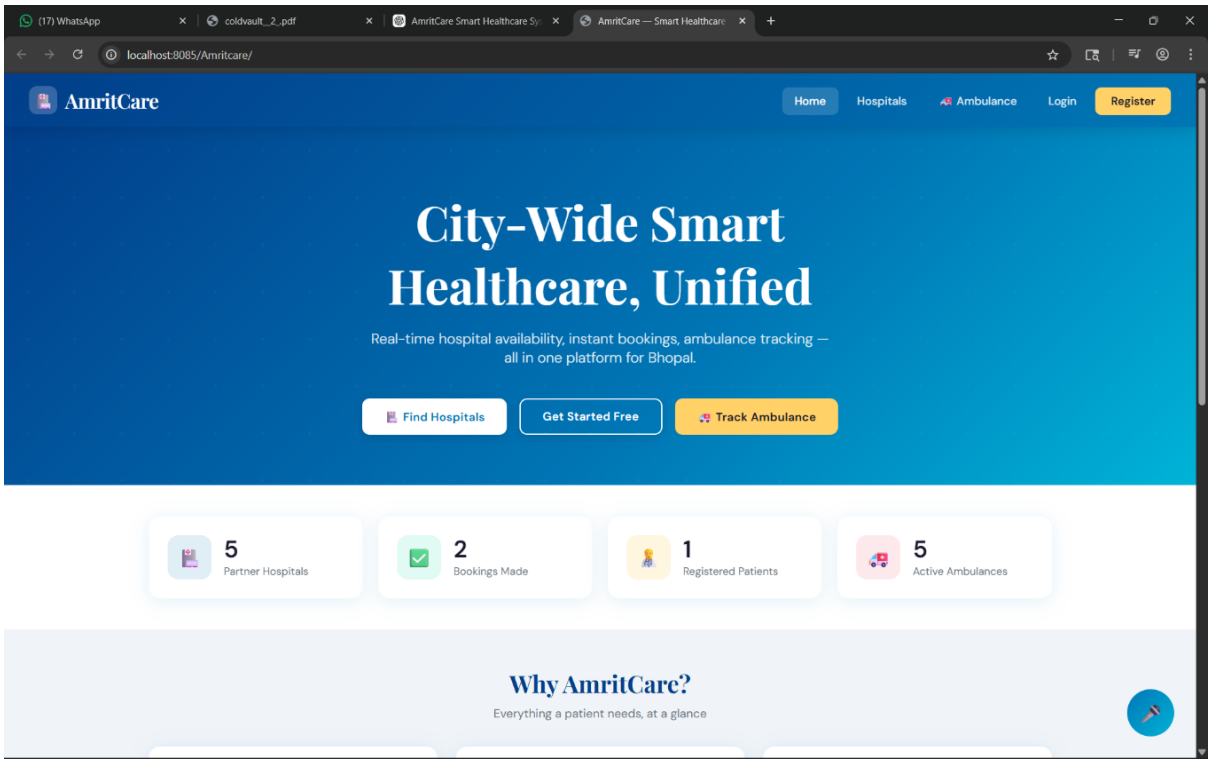

Figure A.2: Home Page of the System

A.3 Patient Interface

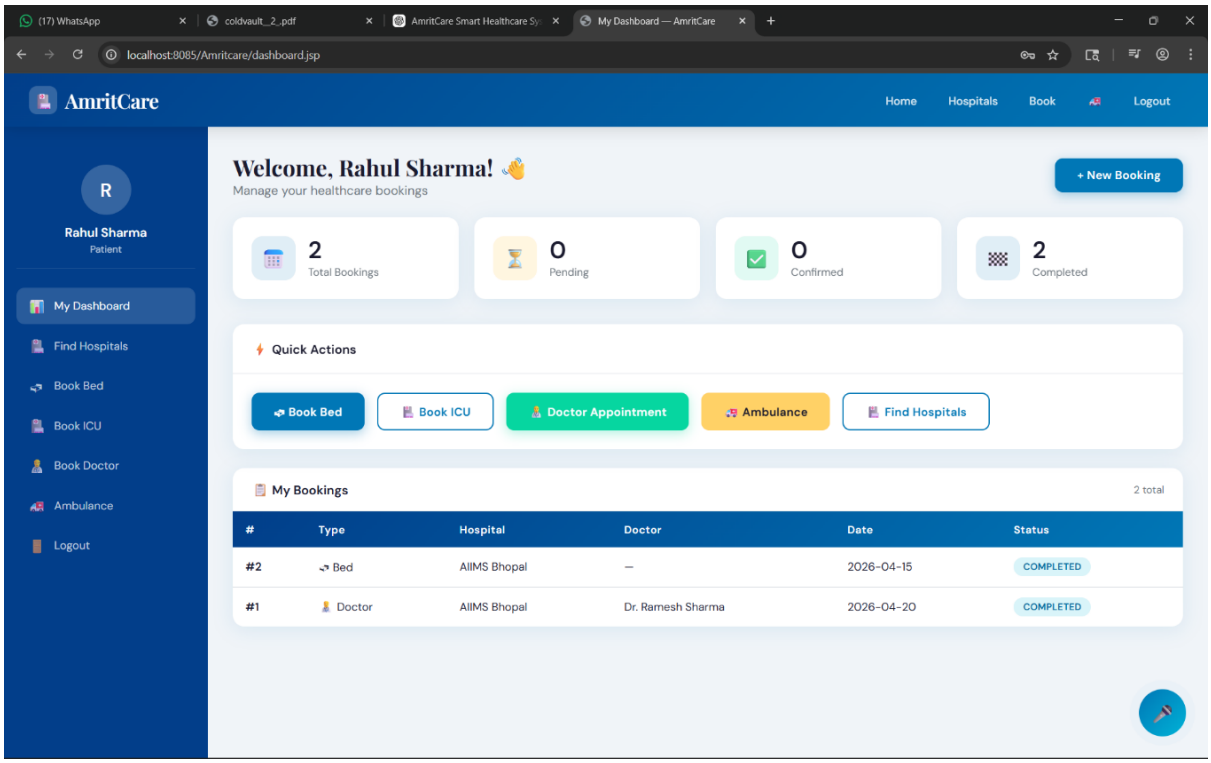

Figure A.3: Patient Interface

A.4 Hospital Interface

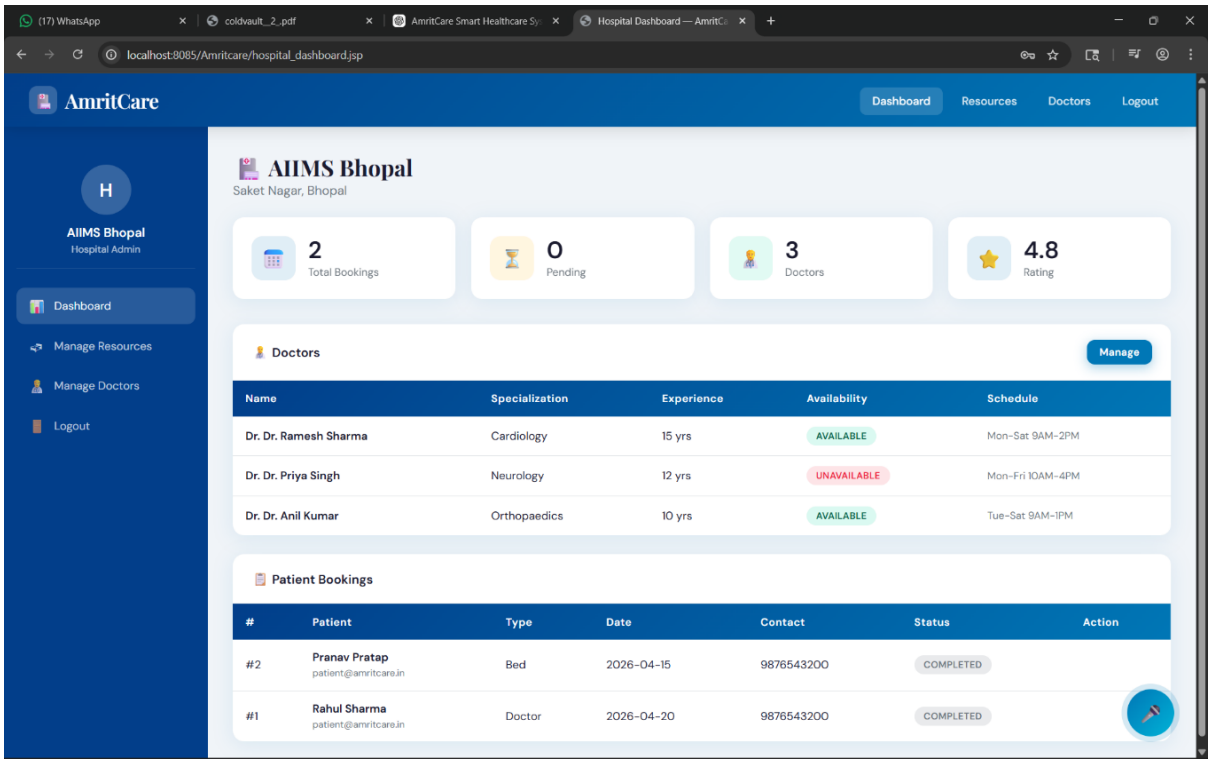

Figure A.4: Hospital Interface

A.5 Find Hospital Dashboard

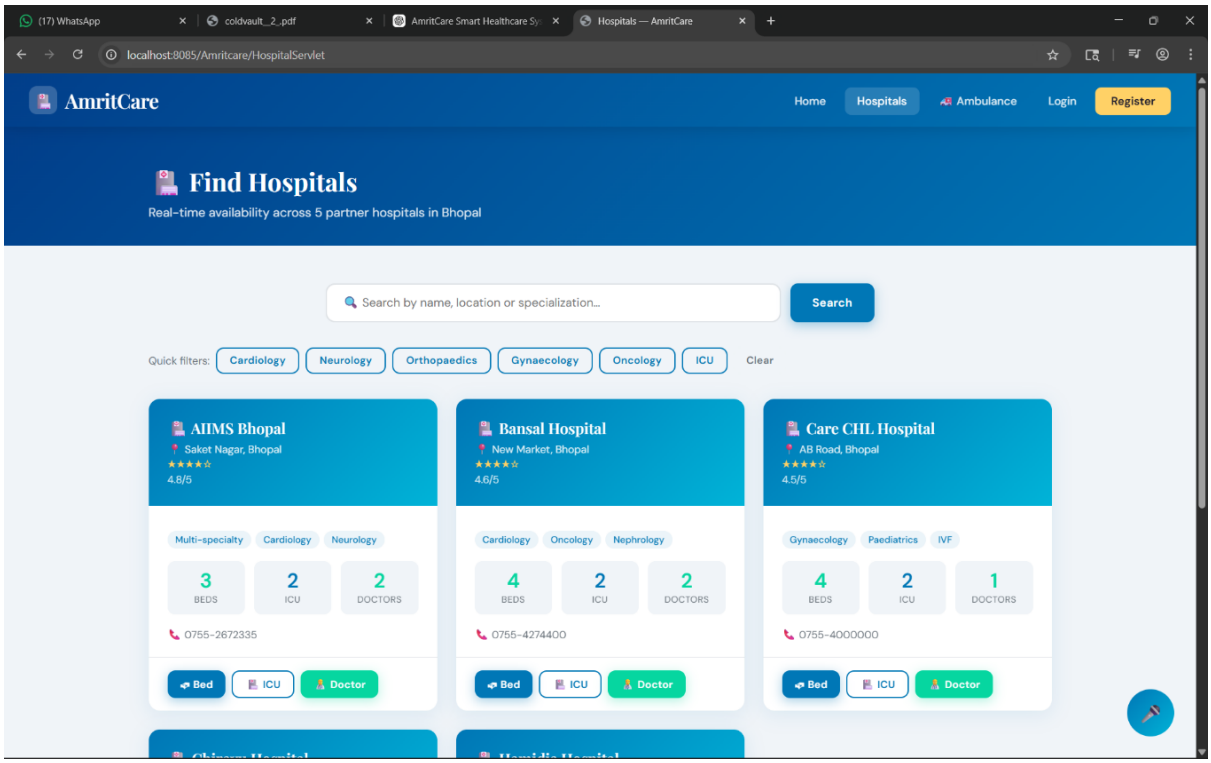

Figure A.5: Find Hospital Dashboard

A.6 City Admin Interface

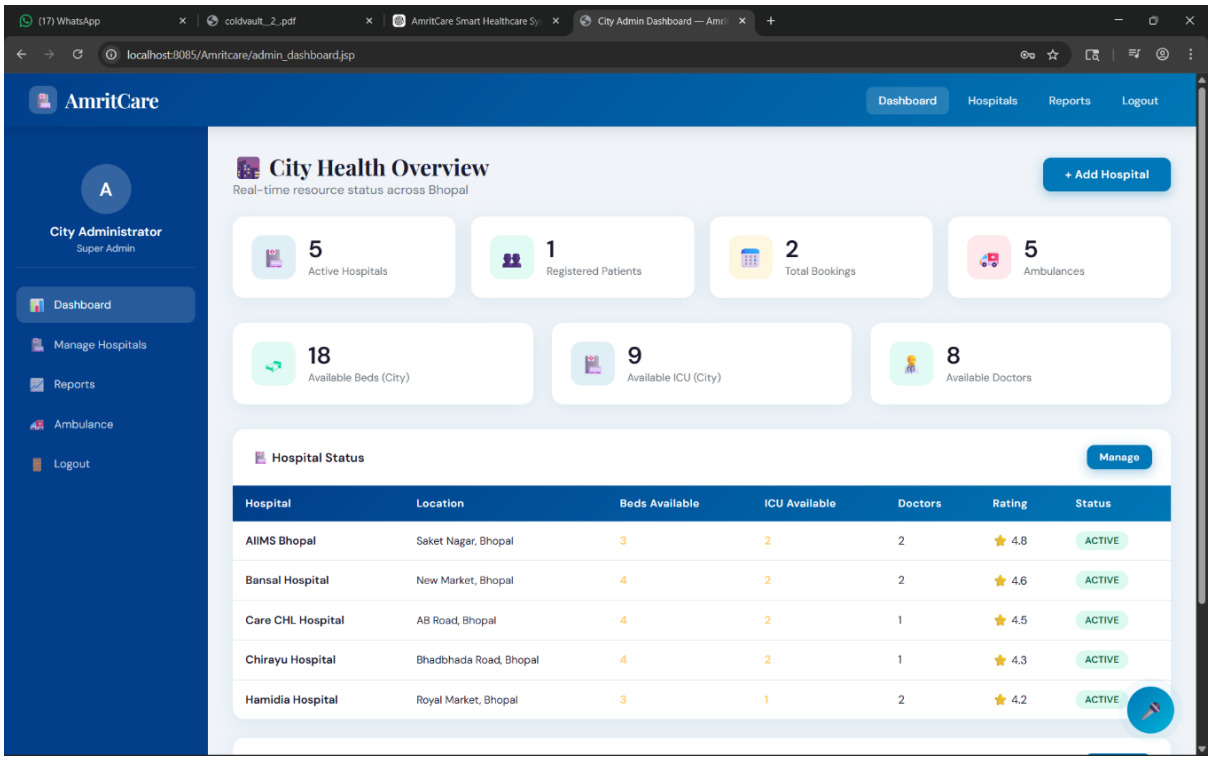

Figure A.6: City Admin Interface

Chapter B Project References / Existing Systems

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[2] Oracle Corporation, "Java Servlet Technology," Oracle Java EE Documentation. [Online]. Available: https://docs.oracle.com/javaee/7/tutorial/servlets.htm. [Accessed: Apr. 2026].

[3] MySQL AB, *MySQL 8.0 Reference Manual*. Oracle Corporation, 2023. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/.

[4] Apache Software Foundation, "Apache Tomcat 9.0 Documentation." [Online]. Available: https://tomcat.apache.org/tomcat-9.0-doc/. [Accessed: Apr. 2026].

[5] W3C Web Platform Working Group, "Web Speech API Specification." [Online]. Available: https://wicg.github.io/speech-api/. [Accessed: Mar. 2026].

[6] Chart.js Contributors, "Chart.js Documentation v4.4." [Online]. Available: https://www.chartjs.org/docs/latest/. [Accessed: Apr. 2026].

[7] R. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.

[8] Ministry of Health and Family Welfare, Government of India, "National Digital Health Mission Blueprint." New Delhi, 2020. [Online]. Available: https://abdm.gov.in/. [Accessed: Feb. 2026].

[9] A. Silberschatz, H. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill Education, 2020.

[10] D. K. Deni and F. Y. Ferida, "Usability testing of information system websites," *Journal of Technology and Applied Industrial Management*, vol. 2, no. 1, pp. 41–52, 2023.